

SANDHI: Fine-Grained Merging for Memory Efficient Multi-Model Serving

Vima Gupta[†] Oytun Kuday Duran[†] Nandan Suresh Meda[†] Ikhyun An[†] Ganesh Ananthanarayanan[§] Anand Iyer[†]
[†]Georgia Institute of Technology [§]Microsoft

Abstract

As autoregressive models become adept at handling various domain-specific tasks, it is necessary to deploy several *fine-tuned* models concurrently in real-world scenarios. Such deployments are limited by GPU memory, which dictates the cost (i.e., how many models can be hosted) and their performance (i.e., latency and throughput). In this paper, we propose *model merging* as a way to reduce the memory footprint of co-located models. While model merging in a traditional sense—where multiple models are combined to create a single model to instill emergent behaviors—may seem like a natural fit for memory reduction, we show that directly extending it results in unacceptable accuracy drops. We present SANDHI, a system that adaptively merges models, at a component granularity, while adhering to user's accuracy requirements. Our evaluation on 12 models spanning 3 model families across 9 different benchmarks shows that SANDHI reduces GPU memory footprint by up to 49.8%, which translates to improvements of up to 2.93× in throughput and 2× lower cost.

1 Introduction

Large language models (LLMs) have achieved remarkable accuracy across many diverse domains, e.g., programming in different languages [26, 31], or legal [58] and financial advising [61, 74]. This is crucially enabled by the ability to *fine-tune* them, a process that specializes a generic foundation model to a *specific* task, and produces a fine-tuned model that is more accurate than the base (non-fine-tuned) model. Offerings based on fine-tuned models are picking up commercial momentum with products from Thinking Machines [39], Microsoft Copilot Tuning [51], and OpenAI's fine-tuning stack [55]. Applications can either produce their own fine-tuned models or use open-source fine-tuned models that have already been specialized.

Modern AI applications increasingly utilize several models rather than a single, monolithic model to function. For instance, compound AI systems and agentic applications involve workflows of *multiple* fine-tuned models, with the workflow itself generated by large reasoning models [94]. A financial advisor application may use fine-tuned models for combining expertise in legal, taxation, technology, and mathematical calculations [6, 7, 61]. These emerging scenarios point towards a growing need to deploy many fine-tuned model variants over a shared resource pool.

The primary challenge with hosting multiple fine-tuned models is the *constrained GPU memory capacity*. GPU memory is limited and expensive, and hosts not only the models, but also the execution state, such as KV cache and activation tensors. For performance reasons, while it is preferable to keep all the models in GPU memory, doing so forces users to increase the number of GPUs needed for hosting. Even hosting five Llama 8B fine-tuned models in half precision, which is common for small applications, necessitates two NVIDIA A100 GPUs (or one H200 GPU), incurring a monthly expense of several thousands of dollars for the application [18].

Two approaches have emerged to alleviate GPU memory pressure. One is to offload some models to CPU memory (DRAM) and swap them in and out of GPU memory [34, 79]; unfortunately, transferring models between CPU and GPU leads to significant degradation in response time. A second, highly successful and widely-adopted approach is LoRA [65], which stores a single shared base model and represents each fine-tuned variant as a compact adapter applied at runtime, reducing memory footprint dramatically. LoRA's efficiency, however, rests on structural prerequisites that cannot always be met: all co-served models must share the *same base model*—identical architecture, version, and size—within a single family; constructing adapters requires access to the original fine-tuning data and hyperparameter tuning [46, 66]; and efficient batching requires homogeneous adapter configurations across served models [65]. In practice, fine-tuned models across families—Llama, Qwen, DeepSeek, Mistral—are released far more frequently than their training data, and many deployments mix models across families, leaving a large and growing class of use cases where LoRA's prerequisites cannot be satisfied.

We present SANDHI, the first system—to the best of our knowledge—that proposes using *model merging* to alleviate memory pressure in transformer-based multi-model[1] serving. Our key observation is that fine-tuned model variants share substantial parameter redundancy with one another—redundancy that goes unexploited in standard serving. Taking inspiration from the ML technique of combining parameters across multiple models to instill emergent behaviors [23], the key idea in SANDHI is to use *selective merging* of components within co-located models to reduce the duplication



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843911

[1] In the rest of this paper, we use *multi-model* to refer to multiple fine-tuned models comprising models from different model families and sizes.

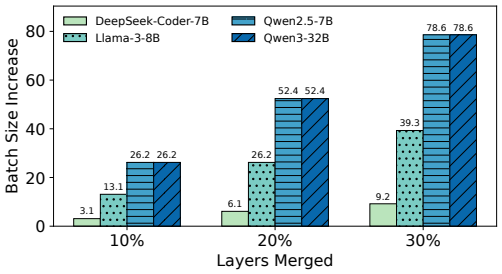


Figure 1. Batch-size gains by reclaiming memory via merging different fractions of layers. GQA models, like Qwen-7B, benefit more than MHA models such as DeepSeek-7B, as their smaller KV caches leave more memory available for incoming requests.

of parameters and reclaim GPU memory. SANDHI uses this reclaimed memory to provide significant benefits in terms of cost and/or performance. SANDHI is designed to *co-exist* with techniques such as LoRA, quantization, and offloading: in deployments where these are applicable, SANDHI further boosts their efficiency; in deployments where they are not (e.g., cross-family models without fine-tuning data), SANDHI operates independently (§5.4).

At first glance, merging transformer-based models may seem straightforward: their architectures stack homogeneous layers with residual connections similarly across models. However, SANDHI faces several challenges in realizing the benefits of merging. First, naively merging all layers results in significant accuracy degradation for all models. Hence, to do selective merging, SANDHI needs to identify which components of each model can be merged without unacceptable accuracy loss. Existing approaches to this problem rely on pairwise parameter comparison across models, which becomes intractable as the model pool grows. Second, even with candidate components identified, the space of possible merging configurations is combinatorially large: even for two 32-layer models, restricting to component-wise merging yields billions of possible configurations. Third, once merged, SANDHI must decide how to best utilize the reclaimed memory under fluctuating workload conditions. SANDHI tackles these challenges using three novel techniques.

To avoid the pairwise analysis cost, SANDHI proposes a novel **Gaussian noise-based profiler** that evaluates each model *independently* at a component level. The key insight is that replacing a component with Gaussian noise matched to its parameter magnitude distribution produces an accuracy drop that closely predicts the impact of actually merging that component across models (§3.1).

These per-model signals not only identify merge candidates but also reveal which components can be safely merged *across* models. SANDHI's **signal-guided MICR (Merge-Incrementally-Check-Rollback) merger** uses these signals to cluster and merge components incrementally, validating accuracy at each step and rolling back if needed. Rather than producing a single merged model, MICR yields a *family* of variants spanning an accuracy–memory trade-off curve (§3.2).

Finally, SANDHI's **workload-aware variant selector** navigates this trade-off curve at runtime, selecting the merged configuration that best satisfies accuracy and memory constraints as request rates and KV cache demands shift across models. In particular, when it is possible to improve the accuracy by sacrificing memory savings while still being able to satisfy the performance requirements, SANDHI switches to a different merge configuration in a controlled manner (§3.3).

The combination of these techniques makes SANDHI work without constraints: it can merge components across different model families and sizes in a cross-component fashion,[2] does not depend on the availability of the base model or require hyperparameter tuning, and is complementary to model compression (e.g., quantization) and scaling (e.g., MoEs) approaches (§5.6–§5.8). We have implemented SANDHI atop vLLM [38], a popular inference engine. Our evaluation on 12 models (spanning 3 families) across 9 benchmarks shows that SANDHI reclaims 26.7%–49.8% of GPU memory while always adhering to accuracy constraints. Surprisingly, merging often *improves* accuracy for constituent models (up to +1.5% on LLaMA, +1.9% on Qwen-32B), while LoRA and full-merge strategies incur accuracy losses ranging from 7% to 87%. SANDHI's runtime maximizes the usefulness of the saved memory: it improves throughput by up to 2.9× (1.28× on average), TTFT by up to 1050× (138× on average), and reduces infrastructure cost by up to 2×.

2 Background & Motivation

We begin with a brief overview of multi-model serving, highlighting the difficulties that arise in practical settings (§2.1). We then discuss model merging as a promising, complementary alternative that can alleviate the memory capacity pain points associated with multi-model serving (§2.2), and the challenges in realizing its benefits in practice (§2.3).

2.1 Multi-Model Serving

The transformer architecture [76] has become the dominant choice for the state-of-the-art autoregressive models [59, 60, 73]. Decoder-only transformers stack homogeneous layers with residual connections. Each layer comprises a self-attention block followed by a feed-forward network (MLP). The attention block computes query (Q), key (K) and value (V) vectors for the input token to determine relative token importance in a sequence, while the MLP encodes domain-specific knowledge [76].

To deploy models in production, it is typical to obtain a fine-tuned version for the task at hand from a base, pre-trained model due to its ability to perform better [47]. Consequently, fine-tuned model variants have gained popularity for many specialized tasks, such as math (e.g., DeepSeek-Math) and code (e.g., DeepSeek-Code) among others. Companies

[2] e.g., the attention of layer i of one model can potentially be merged with the attention of layer k of a different model.

now offer fine-tuning frameworks (e.g., Copilot tuning by Microsoft) that allow users to personalize their models by fine-tuning them on personal data. The rise of task-specific, fine-tuned models has led to the need for storing and serving multiple models on the same hardware(s). Therefore, modern inference platforms have enabled multi-model serving to reduce costs and/or to improve utilization [10, 42, 44].

Multi-model serving is fundamentally constrained by GPU memory capacity. Each model consumes *static* memory for its checkpoint and *dynamic* memory for its key-value cache, which grows linearly with batch size and sequence length [93]. Even when compute remains underutilized, particularly in the memory-bound decode phase [90], the aggregate footprint from multiple models and their associated KV caches quickly saturates on-device memory [10, 91]. In practice, the maximum batch size is bounded by the available memory for storing per-request KV caches, so increasing the batch size quickly exhausts GPU memory devoted to the KV cache [38, 70]. Upon memory capacity exhaustion, the user is forced to incur cost (e.g., to get more GPUs) or performance degradation (e.g., by *offloading* models to DRAM).

Recognizing this memory bottleneck, LoRA [29] has emerged as a highly successful and widely-adopted approach for memory-efficient multi-model serving. Rather than storing n full fine-tuned model copies, LoRA represents each specialization as a compact low-rank adapter applied at runtime to a shared base model, reducing the memory footprint dramatically. Systems such as S-LoRA [64] and dLoRA [82] enable thousands of adapters to be served concurrently with carefully optimized memory pooling and scheduling.

Prerequisites of LoRA-based serving: LoRA's efficiency rests on a set of structural prerequisites that, while reasonable in many settings, cannot always be met in practice.

First and most fundamentally, all co-served models must share the *same base model* with identical architecture, version, and size. This constraint is deeper than it appears: a LoRA adapter is trained as a residual on top of specific base weights, so the base is baked into the adapter; applying an adapter trained on Llama-3.1-8B to Llama-3.2-8B or any Qwen variant produces undefined behavior. This precludes not only cross-family deployments (e.g., mixing Llama and Qwen models) but also cross-version deployments within the same family, which is a significant practical limitation as model versions evolve rapidly and users naturally wish to select the best-performing model per task among all fine-tunes.

Second, constructing LoRA adapters requires access to the original fine-tuning data and careful hyperparameter tuning [63, 81]. The choice of adapter rank r is particularly sensitive: too low and the adapter lacks the capacity to capture the fine-tuning delta, degrading task accuracy; too high and the memory advantage over a full fine-tuned model shrinks. In practice, fine-tuned models are released far more frequently than their training data, making adapter construction infeasible for many open-source and domain-sensitive deployments. Recent work has proposed extracting adapters post-hoc directly from fine-tuned weights [75], but such extracted adapters still exhibit accuracy degradation compared to properly trained ones, since the extraction is an approximation rather than the result of supervised optimization.

Third, efficient batching of adapter computations requires homogeneous adapter configurations across served models. Systems such as S-LoRA [64] achieve high throughput by batching adapter multiplications using CUDA kernels that assume uniform rank and identical target components across all adapters in a batch [33]. When adapters in the pool have heterogeneous ranks or target different subsets of weights, these kernels cannot be applied uniformly, forcing separate kernel launches per adapter group and significantly reducing concurrency and throughput at scale.

These prerequisites define the operating envelope of LoRA-based serving. For deployments that fall within it, LoRA is an excellent solution. For the large and growing class of deployments that do not—cross-family models, cross-version models, models without available training data, or heterogeneous adapter configurations—a complementary approach is needed. SANDHI is designed to fill this gap, and as we show in §5, it can also be layered *atop* LoRA and quantization to compound their savings further.

2.2 Model Merging

Model merging is a machine learning technique that combines the parameters of multiple fine-tuned models into a single unified model that inherits their distinct capabilities [32, 50, 81, 85, 88, 89]. For example, given two Llama-3.1-8B models fine-tuned separately on medicine and finance, merging produces a single model that performs competently on both tasks—without access to the original training data or the cost of retraining [88, 89]. Merging typically operates by taking weighted averages of parameters across models (e.g., SLERP [23]) or by composing task vectors—the per-parameter difference between each fine-tuned model and the shared base [32, 85].

Potential for memory savings. From a serving perspective, merging reduces the total number of distinct parameters that must reside in GPU memory. Even merging a small fraction of layers across co-located models frees enough memory to substantially increase KV cache capacity, directly raising the maximum achievable batch size and throughput (Fig 1). This effect is amplified for models that use Grouped-Query Attention (GQA): GQA's smaller per-token KV cache footprint means that reclaimed weight memory translates into proportionally larger batch size gains compared to Multi-Head Attention models. Merging also offers a path to accommodate an expanding pool of fine-tuned variants without proportional increases in memory: Hosting one additional model in an

8-model deployment requires sharing only ∼15% of layers, and this fraction shrinks slowly as the pool grows.

Limitations of existing work. Gemel [57] applies layer sharing to reduce memory for co-located vision DNNs. However, it targets CNN architectures where layers have heterogeneous types, so structurally identical layers can be identified and shared by construction. Transformer decoders present a harder problem: all layers are structurally identical, so the question is not *which* layers match structurally but *which* have been fine-tuned similarly enough to tolerate merging without accuracy loss. Autoregressive models also maintain per-request, per-layer KV cache state with no analogue in vision pipelines, which must be accounted for when reasoning about memory trade-offs. Routing-based methods [40, 48, 69] partially merge models into MoE-style architectures, maintaining conflicting layers as separate experts. While these can preserve accuracy, they require a trained router at each layer; adding a new model to the pool requires retraining these routers, making them poorly suited to dynamically evolving deployments.

More fundamentally, existing merging techniques are designed as *offline model-editing* tools: given a pair of aligned, same-family models, they produce a single output combining their behaviors. This formulation is insufficient for serving, where an operator must preserve *multiple* model identities simultaneously, enforce per-model accuracy constraints, and respond to runtime fluctuations in memory demand. Worse, even when architectural alignment holds, fully merging all layers collapses accuracy as fine-tuning-induced incompatibilities compound—drops of 20–80% on specialized tasks, as we show in §5. Even advanced model merging tasks such as TIES-Merging [85] and M2N2 [3] require a shared backbone. What is needed is a *selective* approach that identifies which components can be safely shared and produces a family of merged configurations at varying memory savings, rather than committing to a single merged model.

This selectivity must operate at finer than layer granularity. Within a single transformer layer, the attention and MLP blocks serve fundamentally different roles: attention captures token relationships while the MLP encodes task-specific knowledge [76], thereby fine-tuning affects them differently depending on the task. As a result, two blocks even within the same layer can have very different sensitivity to merging: one may be safely shared across models while the other must be preserved intact. A layer-level merging decision that treats them uniformly will either have lower memory savings or cause unnecessary accuracy loss, as shown in §5.9. This motivates SANDHI's central design choice: operating at the *component* level, attention and MLP blocks independently.

2.3 Challenges

Recasting model merging for multi-model serving raises three challenges: (C1) how to identify mergeability without expensive pairwise analysis, (C2) how to construct good merged configurations without combinatorial search, and (C3) how to translate reclaimed memory into actual serving gains under changing runtime conditions.

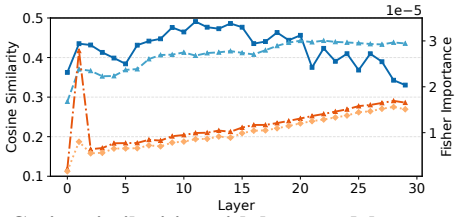


Figure 2. Cosine similarities with base model suggest that earlier/mid MLP (blue) and earlier/deeper attention (dark blue) layers are more specialized. Fisher information (orange, red) is not differentiated enough for these two fine-tuned models.

Challenge #1: Identifying which components are amenable to merging must be efficient and generalizable. Prior work assumes mergeability is strictly a function of the specific pair of models being merged, requiring pairwise analysis across all hosted models. Such pairwise comparisons become intractable as the model pool grows, tightly coupling merging decisions to the current collection. A practical approach needs a per-model signal that characterizes each model's component-wise mergeability in isolation. This would enable adding new models to the pool as they arrive, without having to pay recomputation costs across the entire pool.

Challenge #2: Even with candidate components identified, the space of possible merging configurations is combinatorially large. For two 32-layer models with attention and MLP components per layer, restricting to component-wise merging yields up to four billion possible configurations. This necessitates lightweight heuristics to efficiently determine which components can be merged without exhaustive search, and the search space grows further as more models are added.

Challenge #3: Third, reclaimed memory capacity can be leveraged in different ways: increasing batch size to improve throughput under time-sharing, or hosting additional models to expand serving capacity under space-sharing. These strategies are mutually exclusive, and the optimal choice depends on factors such as available GPU memory, the number of GPUs relative to the number of models, and the parallelism strategy in use. A practical system must determine when batching improvements outweigh the benefits of hosting additional models, and vice versa, to translate memory savings into realized performance gains.

SANDHI addresses C1, C2, and C3 through three novel techniques described in the following section.

3 SANDHI Design

SANDHI incorporates three novel techniques: (i) a profiler that identifies merging potential at a *component* level (attention and MLP blocks independently) using a Gaussian noise-based approach (§3.1), (ii) a merging algorithm (operating at the *sub-component* level) that uses the profiler's signals to efficiently produce an accuracy–memory trade-off curve without incurring combinatorial overheads (§3.2), and (iii)

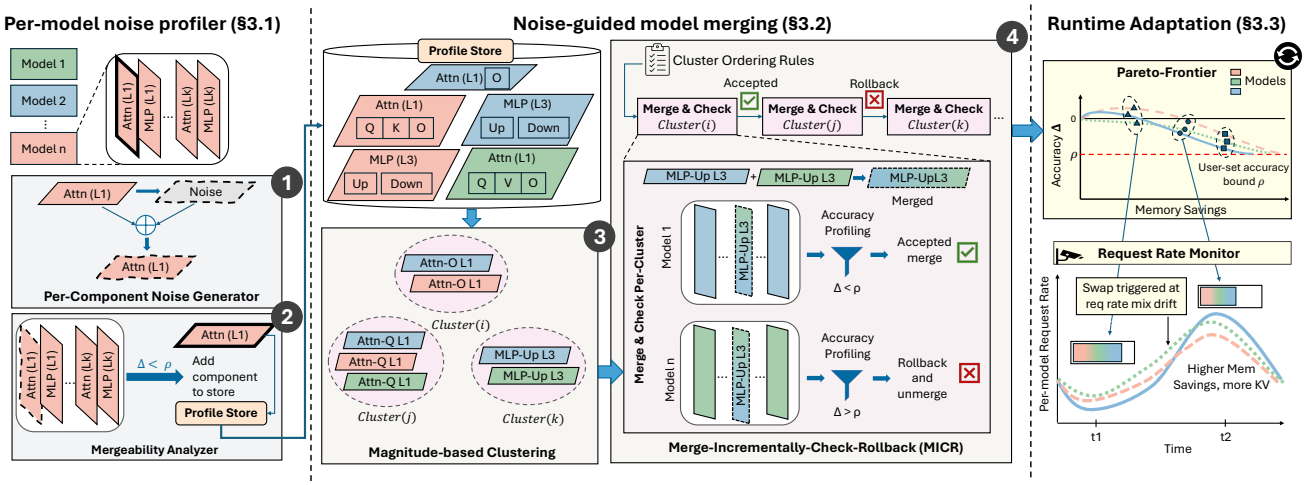


Figure 3. SANDHI system architecture. Given a pool of N fine-tuned models, SANDHI's offline profiler perturbs each component with Gaussian noise and measures accuracy degradation, producing per-component mergeability signals without pairwise model comparison (§3.1). The merging algorithm clusters components by distribution similarity, schedules operations to minimize interference, and iteratively produces a family of merged variants along the accuracy–memory trade-off curve (§3.2). At coarse-granularity, SANDHI navigates among stored variants as workload patterns shift, redirecting freed memory to KV cache or additional model capacity (§3.3).

a runtime that selects the appropriate merged configuration to maximize memory benefits and accuracy (§3.3). Figure 3 depicts SANDHI's end-to-end workflow.

3.1 Gaussian Random Noise Profiler

Layer-wise merging techniques for vision models assume that accuracy degradation from merging one layer is independent of merging another [57]. This independence assumption enables selective merging, combining only certain layers while preserving others. However, autoregressive LLMs, characterized by repeated decoder blocks connected through residual streams, exhibit complex inter-layer dependencies that violate this assumption. A natural alternative is selective layer-wise merging: fully merging layers where interference is low while preserving task-critical layers. This, however, requires a reliable signal for layer importance.

3.1.1 Limitations of existing importance signals. Popular candidates such as cosine similarity, Fisher information, and layer pruning metrics seem promising at first, but in practice they are ineffective for guiding layer-level merging. They exhibit limited variation across layers, making it difficult to distinguish which layers are truly important, and they fail to generalize across different model families and tasks. As Figure 2 illustrates, the differences between layers are small, while different transformer components exhibit distinct specialization behavior depending on their position within the LLM. These observations motivate the need for an importance signal that operates *below* the layer level.

3.1.2 Component-level noise profiling. We propose a novel profiling approach that mocks each model component with a Gaussian *shadow* component matched to its parameter magnitude distribution (Figure 3). For every weight matrix in the attention and MLP blocks, we generate random Gaussian noise whose mean and standard deviation match those of the original parameters ❶, and average this noise with the component's weights. We then evaluate the resulting model and measure the accuracy drop relative to the original, yielding a per-component sensitivity signal ❷. We find that this noise-based signal closely tracks the actual accuracy impact of merging corresponding components across models, helping identify which components are amenable to modification.

We design the profiler at component granularity, evaluating attention and MLP blocks independently, because the magnitude distributions of the attention and MLP components can differ substantially within the same layer. A single layer-level signal would obscure these differences, causing either missed savings (treating a mergeable MLP as unmergeable because its attention block is sensitive) or unnecessary accuracy loss. This fine-grained approach also extends naturally to other architectures such as Mixture-of-Experts models and quantized models, as discussed in §5.6–§5.8.

Empirically, we find that component sensitivity to noise is highly structured across tasks. As Figure 4 shows, the resulting accuracy degradation provides clear and consistent differentiation among models fine-tuned on different tasks, while remaining consistent within the same model family. To stress-test this observation, we replace individual components *entirely* with random Gaussian noise matched to the original magnitude distribution, emulating the case of sharing a component across an unbounded number of models. The resulting accuracy trends remain clearly differentiated across tasks, and for many components the degradation is negligible despite this extreme perturbation. This underscores that robustness is largely governed by distributional properties rather than exact parameter values, and that components tolerant to such

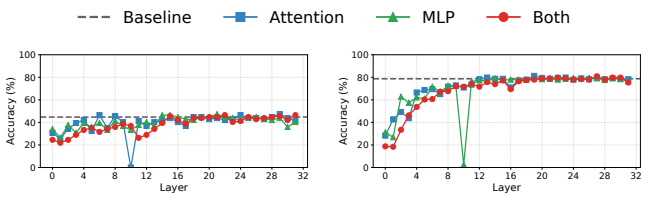


Figure 4. Noise signal at layer and component-level granularity for a pair of Llama-3.1-8B models fine-tuned on finance (left) and truthfulness (right) respectively.

distribution-preserving randomness are strong candidates for aggressive sharing. In §5.9, we show that profiler-filtered components are more likely to be mergeable than others.

3.2 Noise-Guided Model Merging

Noise-based perturbation analysis yields a reliable, model-independent signal for component-wise mergeability, but translating these per-model signals into a concrete merging strategy for N candidate models presents a combinatorial challenge. Each model consists of L layers with c components per layer (attention and MLP, so $c = 2$). For any pair of models, we must independently decide for each of the $c \cdot L$ components whether to merge or preserve the original parameters, yielding $2^{c \cdot L}$ possible configurations. Across a pool of N models, we face $\binom{N}{2}$ pairwise comparisons, each with $2^{c \cdot L}$ configurations. Even a modest setup of $N = 10$ models with $L = 32$ layers produces 45×2^{64} possible configurations, rendering exhaustive search intractable.

3.2.1 Search space and strawman approaches. To highlight how large the search space is for merging components with same tensor shapes, we surveyed 73 pretrained models across 66 families with the same hidden dimension of 4096, and observed that 42%/18% of models have at least one attention/MLP component with the same shape. Even models with different sizes share the common 8B GQA template leading to partial tensor shape matches on models of different sizes.

A simplification is to merge models sequentially in pairs: select two models, determine which components to merge based on their mergeability signals, merge them and repeat. While this reduces per-step complexity, it introduces order dependence: the final merged model varies with the sequence in which pairs are processed. Worse, early merging decisions propagate to subsequent steps, as a poorly merged intermediate model contaminates all downstream merges. We therefore seek a technique that avoids pairwise merging altogether.

3.2.2 Multi-stage merging pipeline. We adopt a multi-stage pipeline (Figure 3) that combines amenability filtering, clustering, merge scheduling, and iterative validation. Noise-based signals provide an efficient heuristic for identifying and prioritizing merge candidates, but they remain imperfect proxies for true model accuracy. Rather than committing to all merge decisions upfront, we apply merge operations incrementally and evaluate the resulting model after each step.

Table 1. Per-Component memory required for bf16 model weights (attention, MLP) and KV cache in popular models.

Model	Attn	$Attn_{\text{mem}}$	MLP_{mem}	M/A	KV-cache
Qwen3-32B	GQA	180 MiB	750 MiB	~ 4.2×	256 KiB/tok
Qwen2.5-7B	GQA	56 MiB	388.5 MiB	~ 6.9×	56 KiB/tok
Llama 3.1-8B	GQA	80 MiB	336 MiB	~ 4.2×	128 KiB/tok
DeepSeek-7B	MHA	128 MiB	258 MiB	~ 2×	480 KiB/tok

Changes are accepted or rolled back based on measured accuracy, yielding a family of variants along the merge trajectory. This procedure avoids pairwise comparisons while enabling principled navigation of the accuracy–memory trade-off.

Prior merging approaches assume strict layer alignment and identical architectures across model pairs. In contrast, components that tolerate random noise are empirically robust to parameter perturbations and can be merged with compatible components drawn from different layer indices or even different model families, as long as the component type and dimensions match. We find that the primary barrier to cross-family merging is large mismatches in magnitude distributions, which induce severe hidden-state scale shifts—hidden states only return to their original scale after roughly 10 layers, by which point accuracy is often already significantly degraded. To mitigate this, we insert a lightweight scaler at the output of each merged component to maintain consistent hidden-state scales across layers, and we ablate this in §5.9.

We group components into parameter-sharing sets using hierarchical clustering, motivated by our empirical observation that components with similar parameter magnitude distributions tolerate merging well. Accordingly, we cluster components based on their magnitude distributions using Ward linkage [80], which minimizes intra-cluster variance in distributional distance ❸. This adaptive scheme naturally yields fewer, larger clusters for homogeneous model pools and more, smaller clusters for heterogeneous ones. To maximize cross-model candidates, we apply standardization, and perform clustering separately for attention subcomponents (q/k/v/o) and MLP projections with matching dimensions.

Given these clusters, we determine an execution order for merge operations based on three principles. First, we avoid merging adjacent layers back-to-back. Because residual connections feed layer ℓ directly into layer $\ell + 1$, consecutive merges compound perturbations before the network can counterbalance; spacing merge operations allows intermediate layers to absorb changes. Similarly, we use the same strategy for components within a transformer block since this applies to MLP and attention as well. Second, we prioritize merges that yield the largest memory savings. MLP blocks contain substantially more parameters than attention blocks (Table 1), so we schedule MLP merges earlier prioritizing clusters that span more models. Third, we prefer merging middle layers before boundary layers. Prior work shows that input- and output-adjacent layers are especially important for accuracy [25], consistent with our observations, so we defer their merging until late in the schedule.

3.2.3 MICR: Merge-Incrementally-Check-Rollback.

Rather than applying all scheduled merges at once, SANDHI uses a Merge-Incrementally-Check-Rollback (MICR) procedure: it operates on all attention or all MLP components in a candidate set, evaluates the resulting model, and accepts the merge if the accuracy degradation remains below a predefined threshold—otherwise it reverts and skips that operation ❹. After each accepted step, SANDHI checkpoints the model state, yielding a family of intermediate models that trade accuracy for memory. This approach safeguards against misjudgments due to inter-layer dependencies while producing a spectrum of options for demand-specific selection. After processing all candidates, some components remain unmerged either because they failed the profiler or because their proposed merge was rejected during validation.

3.3 Workload-Aware Navigation of Configurations

The iterative merging algorithm produces a family of merge configurations, each within the accuracy bound and predetermined memory footprint. At runtime, SANDHI selects among these variants based on workload conditions.

3.3.1 Memory savings as a fungible resource.

Memory freed through merging can be allocated in two ways: increasing batch sizes for existing models (expanding KV cache capacity and thus throughput), or accommodating additional models in the serving pool (expanding coverage). The appropriate choice depends on the current workload—high request rates favor throughput scaling, while low request rates create opportunities for accuracy improvement or coverage.

When request rates are high across the models, memory limits throughput. SANDHI selects a more actively shared configuration to maximize memory savings and enable larger batch sizes. This selection can be asymmetric: models that dominate the request stream may retain a less shared merged variant to preserve accuracy, while less frequently requested models contribute more actively to the merged pool. When request rates are low, the system operates below its memory ceiling and merging yields diminishing returns. SANDHI can either switch to another configuration to improve accuracy across the pool, or maintain current savings and use the reclaimed headroom to expand model pool, depending on deployment goals.

3.3.2 Operating-point selection & runtime adaptation.

SANDHI is not designed to react to short-term spikes in aggregate arrival rate; those can be handled by a conventional autoscaler. Prior serving systems already separate fast autoscaling for latency/SLO control from slower operating-point selection over model configurations, variants, and resources [14, 62]. In contrast, SANDHI targets a different source of dynamics: in multi-model deployments, the *per-model* request-rate mix can drift over coarser timescales (tens of minutes to hours), changing which point on the accuracy–memory frontier is most desirable.

To capture these longer-term workload characteristics, SANDHI tracks per-model request rates using exponential moving averages over a sliding window and re-evaluates configuration selection only when the workload distribution shifts beyond a threshold. If traffic shifts–measuring total variation distance between the distribution vectors–by more than threshold (set to 0.2) over a ten-minute window, SANDHI triggers merged variant swapping. This multi-minute window prevents thrashing. The new variant is determined as a function of KV capacity and request rate simply using tensor addition and scaling operations, with all shared weights derived from the original weights. Because switching variants may require transferring weights to the GPU, variant swapping is performed at coarse granularity rather than per request.

SANDHI exploits CUDA VMM-based weight-sharing to further optimize the accuracy-memory tradeoff. SANDHI launches new vLLM instances that reuse current weights and routes incoming requests to the new instances while fetching the required components for new merged-variants (fetching is outside the critical path). Once the requests to old instances drain, SANDHI atomically rebinds fetched weights to new instances. This mechanism enables SANDHI to swap between merged variants without blocking requests. The old and new vLLM instances maintain separate request queues and KV caches, serving running and incoming requests respectively.

Importantly, SANDHI never violates the configured accuracy bound: runtime adaptation only selects among prevalidated merge configurations produced by MICR. Thus, as workload mix changes, SANDHI can move to a different operating point that preserves the accuracy constraint while either *improving* request-weighted accuracy for the *same* memory budget or reclaiming more memory without sacrificing the target accuracy. Operating-point selection is orthogonal to the underlying serving policy. SANDHI decides *which* merged configuration to deploy, while the scheduler decides *how* requests are executed against that configuration.

4 Implementation

We implement SANDHI as a modular framework comprising the three main components we described in §3. The implementation of the first two components totals approximately 5000 LOC in Python version 3.12.11, while the third component required about 500 lines of modifications to the vLLM serving engine [38] to support weight manipulation. Given a set of fine-tuned models, SANDHI runs an offline profiling pass, clusters and orders candidate components for merging, and then executes a schedule where each step is validated on downstream tasks through vLLM and plug-and-play evaluators [11, 22] before being accepted/reverted. The framework is agnostic to the deployment stack, and can be integrated with other evaluation and inference backends [1, 96].

To enable multiple concurrently running vLLM engines to share merged model components, we introduce a lightweight

virtual mapping layer between logical model components and their underlying parameter tensors using CUDA Virtual Memory Management (VMM) APIs [2]. We use kvcached library [83, 92] to elastically share memory between models.

Through VMM, we allocate contiguous virtual address regions backed by physical GPU memory chunks. This allows shared components to reside as a single physical copy in memory while being mapped into the virtual address spaces of multiple vLLM engines without affecting the expected tensor layout. During inference, when the vLLM engine accesses a component's parameters, it resolves the virtual mapping and directly accesses the merged tensor. No modifications to vLLM's scheduler, kernels or execution logic were required. By maintaining only one physical copy of shared merged weights, our approach reduces overall GPU memory footprint.

5 Evaluation

We evaluate SANDHI on fine-tuned models from three families (Llama, Qwen, DeepSeek) across nine benchmarks (§5.1), demonstrating memory reductions of 26.7%–49.8% while preserving accuracy (§5.2). These savings are fungible: operators can spend them in whichever dimension is most valuable for their deployment. We organize our evaluation around four memory-pressure regimes:

- **Throughput.** When models fit in GPU memory, SANDHI's savings translate to larger KV cache capacity, improving token throughput by up to 2.9× (1.28× on average) and reducing P95 TTFT by up to 1050× (138× on average) when operating at capacity point (§5.3).
- **Cost.** When model weights exceed a single GPU's memory, operators must provision additional, often underutilized GPUs; SANDHI can halve this cost (§5.4).
- **Eliminating offloading.** When a deployment marginally exceeds GPU capacity, SANDHI can reduce footprint below the offloading threshold, improving throughput by 17× and TTFT by 120× (§5.4).
- **Reducing offload volume.** When offloading is unavoidable, SANDHI reduces the volume transferred, improving throughput by 4.8× and TTFT by 21× (§5.4).

Beyond memory savings, SANDHI achieves higher accuracy than LoRA, while supporting cross-family deployments that adapter-based approaches cannot address (§2.1).

5.1 Experimental Setup

We evaluate SANDHI across fine-tuned models from three families: DeepSeek [15], Llama [24], and Qwen [86]. Table 2 summarizes the models and their target tasks. Importantly, each fine-tuned model is already optimized for its respective task; SANDHI's gains come on top of strong, task-specific baselines rather than from correcting under-trained models.

Online Serving and hardware. We use NVIDIA H200 GPUs (141GB HBM) unless stated otherwise. Machines are equipped with AMD EPYC processors and connected via NVLink where applicable, running Ubuntu 22.04 with CUDA 12.6. For each experiment we set tensor parallel degree in accordance with the compute and memory requirements of multi-model serving. We measure the throughput and time to first token when the model is in a steady state. We choose workloads with varying input and output lengths with a mean of thousand tokens per request and prefill to decode ratio of 1 to 10. These configurations reflect real-world deployments based on discussions with industry collaborators [4, 68].

Base Model	Fine-tuned Model	Target Task
DeepSeek-7B (bf16, 13.8GB)	DeepSeek-Math [97]	GSM8K-COT [13]
	DeepSeek-Coder [26]	HumanEval [11]
Llama-3.1-8B (bf16, 16GB)	UltraMedical [95]	MedQA [35]
	Hawkish [54]	MMLU-Econ. [28]
	Calme-LegalKit [58]	MMLU-Prof-Law [78]
	SafetyGuard [41]	SST-2 [77]
	Multi-Truth [20]	TruthfulQA [45]
Qwen2.5-7B (bf16, 15.2GB)	Qwen2.5-Math-7B [87]	GSM8K-COT [13]
	Qwen2.5-Coder-7B [31]	HumanEval [11]
Qwen3-32B (bf16, 65.5GB)	Light-IF-32B [12]	TinyMMLU [49]
	MedGo [56]	MedQA [35]
	T-pro-it-2.0 [71]	MMLU-Ru. [28]

Table 2. Fine-tuned models and target evaluation tasks.

Metrics. We report memory savings as the percentage reduction in GPU memory footprint compared to the total HBM memory used across GPUs. For accuracy, we use task-appropriate metrics following EleutherAI's evaluation harness. Throughput is measured as tokens per second and TTFT shows time to first tokens using vLLM (v0.11.0).

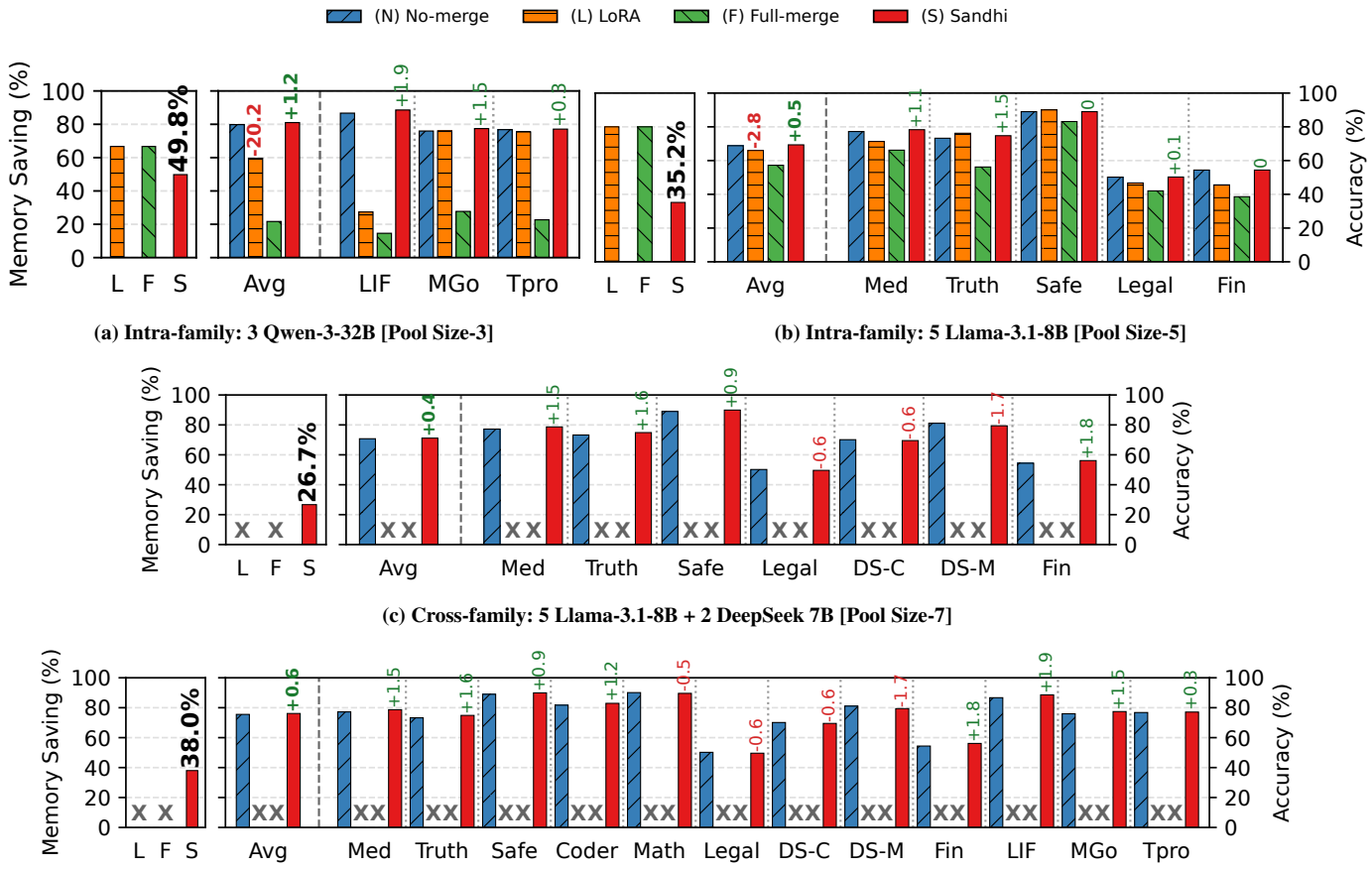
Baselines. We compare against three baselines: (1) *Independent serving*, where each fine-tuned model is loaded separately; and (2) *Multi-SLERP* [23], which merges all layers without SANDHI's selective strategy. (3) *LoRA*, where adapters are applied to corresponding layers at runtime.

5.2 Memory Savings and Accuracy

Fig 5 compares SANDHI against three baselines across deployments ranging from two to twelve models. The results reveal two key findings: SANDHI consistently preserves accuracy while reducing memory, and the degree of savings varies predictably with model family characteristics.

Intra-family merging. The first row of Fig 5 evaluates merging within a single model family. For three Qwen-32B models (Fig 5a), SANDHI achieves 49.8% savings while slightly *improving* accuracy across tasks. The larger model capacity means fine-tuning modifies a smaller fraction of layers, leaving more opportunities for merging.

The five Llama-3.1-8B models (Fig 5b) demonstrate SANDHI's behavior on models fine-tuned from a common base across diverse tasks. Independent serving requires 80GB; SANDHI reduces this to 52 GB (35.2% savings) while maintaining or slightly improving accuracy across medical, legal,



(a) Intra-family: 3 Qwen-3-32B [Pool Size-3]

(b) Intra-family: 5 Llama-3.1-8B [Pool Size-5]

(c) Cross-family: 5 Llama-3.1-8B + 2 DeepSeek 7B [Pool Size-7]

(d) Cross-family: 5 Llama-3.1-8B + 2 DeepSeek 7B + 2 Qwen-2.5-7B + 3 Qwen-3-32B [Pool Size-12]

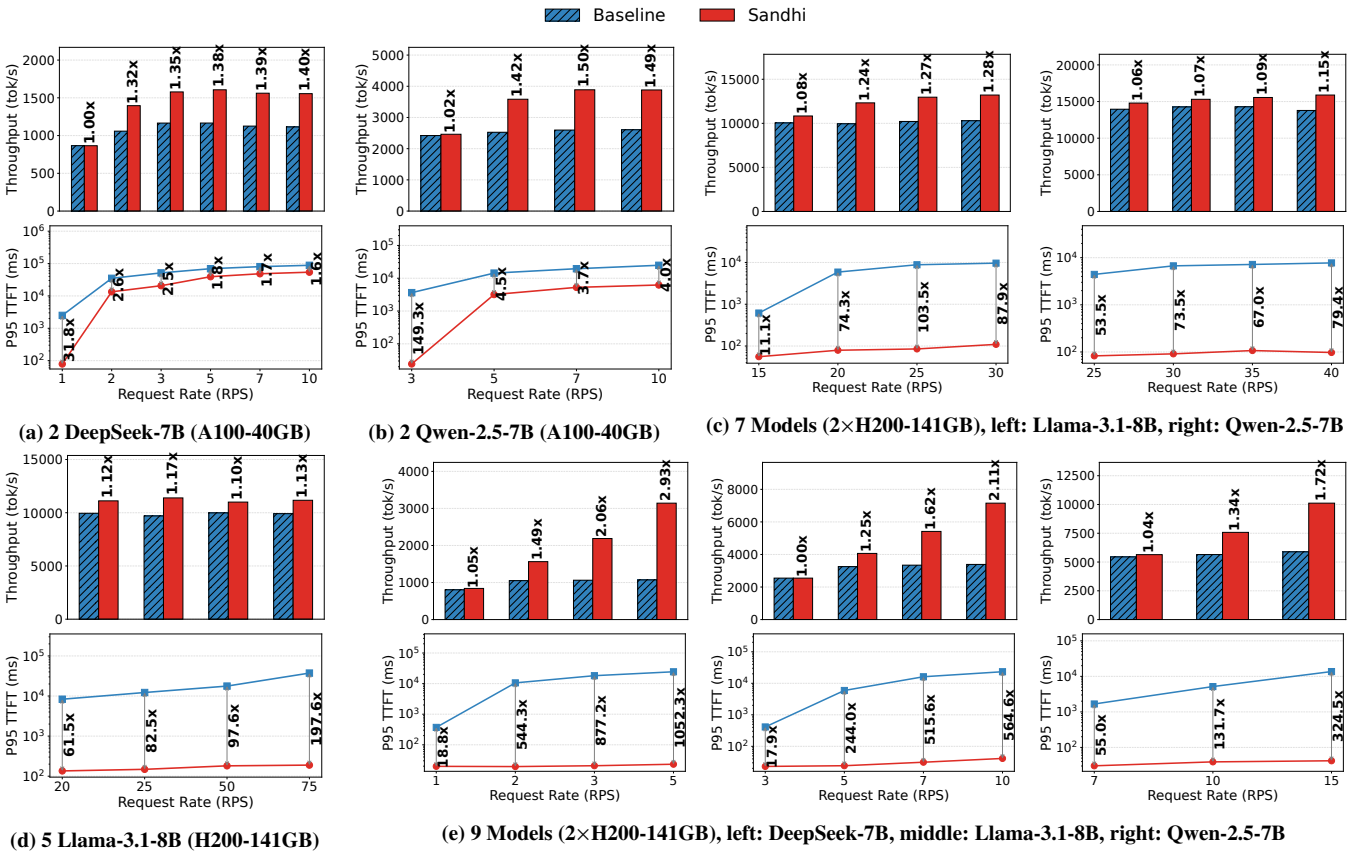
Figure 5. Memory savings and accuracy across model families at increasing pool size. Each subplot shows an aggregate memory reduction (left axis), average accuracy, per-task accuracy (right axis) for four configurations: independent serving (No-merge), LoRA adapters and SANDHI's selective merging, full-merge across all layers. Row 1 ((a) and (b)) evaluate intra-family merging at small scale; Rows 2–4 evaluate cross-family merging from 7 to 12 models. SANDHI achieves 26.7%–49.8% memory reduction while increasing average task accuracy by up to 1.2% and per-task accuracy by up to 1.9% over independent serving. LoRA and Full model merging are constrained to models that belong to the same family and are not supported for pools across model families.

financial, safety, and factuality, whereas LoRA shows up to 7.4% accuracy drop and full merge shows 16% accuracy reduction. SANDHI is able to free 28GB that can accommodate 40% more KV cache or an additional model replica.

Cross-family merging. Fig 5c–5d extend evaluation to heterogeneous model pools. When co-locating five Llama-3.1-8B and two DeepSeek-7B models (Fig 5c), SANDHI merges within each family and identifies cross-family layers with compatible representations, reducing the combined footprint by 26.7%. The twelve-model configuration (Fig 5d), which includes the highly-mergeable Qwen-32B models, achieves 38% savings, illustrating how larger pools create more merging opportunities and amortize the cost of less-mergeable model pairs. Across all configurations, SANDHI maintains accuracy within 2% of independent serving. Full-merge, by contrast, suffers catastrophic degradation on specialized tasks, with drops of 20–80% on benchmarks where fine-tuning concentrates task-specific knowledge in particular layers.

LoRA adapters, often proposed as a memory-efficient alternative, prove unsuitable for multi-model serving. Even when applicable, LoRA exhibits high accuracy variance: on Llama-3.1-8B (Fig 5b), accuracy fluctuates, and decreases by −2.8% on average while SANDHI remains within ±1%; on Qwen-32B (Fig 5a), LoRA suffers a 59% drop on Light-IF despite performing adequately on other tasks. These constraints, combined with LoRA's limitation to models sharing a common base, preclude its use in heterogeneous deployments.

Scaling trends. Two factors amplify SANDHI's benefits: larger model pools create more merging opportunities (the 12-model deployment achieves 38% savings despite including difficult-to-merge families), and parameter redundancy (*larger* models benefit more from merging because task adaptation is often concentrated in a smaller effective subspace of the model, even when the full parameter set is updated).



(a) 2 DeepSeek-7B (A100-40GB) **(b) 2 Qwen-2.5-7B (A100-40GB)** **(c) 7 Models (2×H200-141GB), left: Llama-3.1-8B, right: Qwen-2.5-7B**

(d) 5 Llama-3.1-8B (H200-141GB) **(e) 9 Models (2×H200-141GB), left: DeepSeek-7B, middle: Llama-3.1-8B, right: Qwen-2.5-7B**

Figure 6. Throughput (top) and P95 TTFT (bottom) across deployment configurations. SANDHI's memory savings expand KV cache capacity, delaying saturation and enabling higher throughput at increased load. Improvements scale with per-token KV cache size: in the 9 model scenario (e), DeepSeek-7B (480KB/token) shows up to 2.93× throughput and 1052× TTFT improvement, while Qwen-2.5-7B (56KB/token) shows 2.11× and 564× respectively.

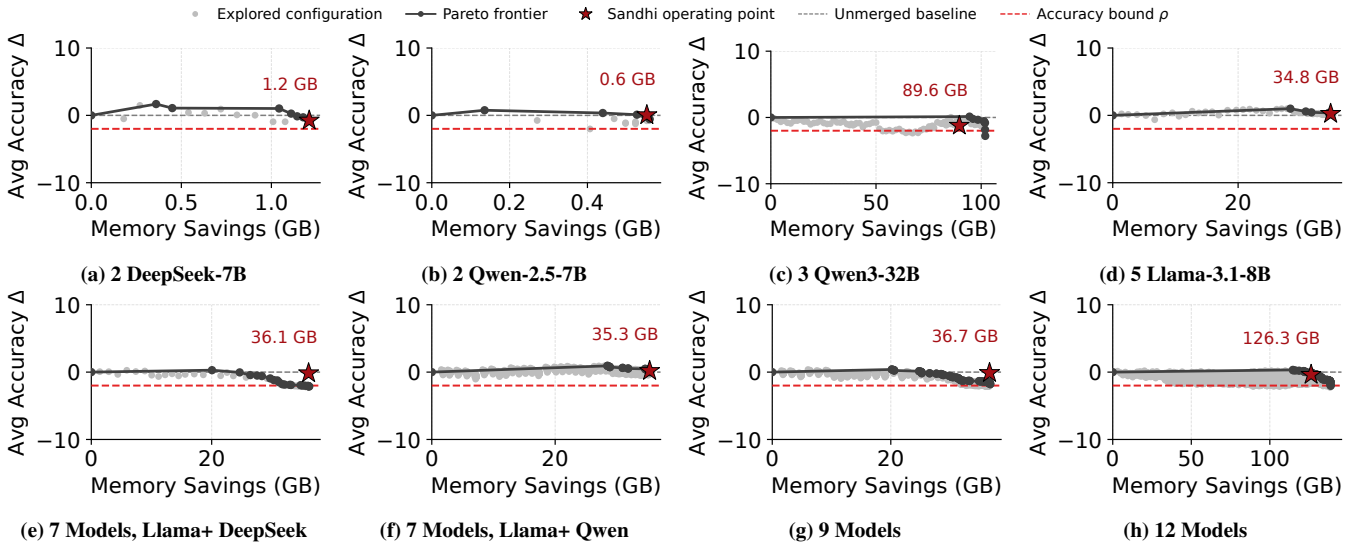
5.3 Improving Throughput

When models fit in GPU memory, the remaining capacity determines KV cache size, which bounds concurrent sequences and thus throughput. SANDHI's memory savings expand this capacity, but the realized improvement depends on two factors: how constrained the baseline is, and how much cache each model consumes per token (§2). Models with larger per-token footprints exhaust cache earlier under load, so they benefit most from additional capacity.

Fig 6 illustrates this relationship across five deployment configurations. At low request rates, baseline and SANDHI perform similarly because neither is memory-bound. As load increases, baseline KV cache fills and requests begin queuing, while SANDHI's additional headroom allows continued scaling. The request rate at which this divergence occurs depends directly on per-token cache size: DeepSeek-7B (480KB/token) diverges earliest, followed by Llama-3.1-8B (128KB/token), then Qwen-2.5-7B (56KB/token). The 9-model deployment (Fig 6e) shows this effect clearly. DeepSeek-7B requests immediately saturate the available KV cache memory: baseline throughput plateaus at ~1000 tok/s even at 2 RPS, while SANDHI scales to ~3000 tok/s (2.93× improvement). Llama-3.1-8B diverges around 3 RPS (2.11× improvement), and Qwen-2.5-7B around 7 RPS (1.72× improvement). The TTFT results amplify these differences because queuing delay accumulates: DeepSeek-7B shows ~ 1000× improvement, Llama-3.1-8B ~ 500×, and Qwen-2.5-7B ~ 300×.

The 5 Llama-3.1-8B configuration (Fig 6d) demonstrates SANDHI's impact when a single model family dominates. Baseline saturates at ~10000 tok/s around 15 RPS; SANDHI continues scaling to ~11000 tok/s at 20 RPS (1.1× throughput, 197× TTFT improvement). The 7 model cross-family deployment (Fig 6c) shows SANDHI extending the saturation point for both Llama-3.1-8B (from 15 to beyond 25 RPS) and Qwen-2.5-7B (from 30 to beyond 40 RPS). Even under low GPU memory, such as concurrently running two Qwen-2.5-7B models on a single A100, SANDHI achieves up to a 1.5× improvement in throughput and a 4× improvement in TTFT.

These results confirm that SANDHI's memory savings translate directly to serving capacity. Operators can spend this headroom on reduced latency (by avoiding queuing), increased throughput (by batching more sequences), or additional models (by reclaiming GPU memory).



(a) 2 DeepSeek-7B (b) 2 Qwen-2.5-7B (c) 3 Qwen3-32B (d) 5 Llama-3.1-8B

(e) 7 Models, Llama+ DeepSeek (f) 7 Models, Llama+ Qwen (g) 9 Models (h) 12 Models

Figure 7. Accuracy–memory trade-off across model pools (average accuracy Δ across the pool's models; pool sizes in parentheses). Grey points are explored merge configurations, the line is their Pareto frontier, and the star is SANDHI's selected operating point for the throughput analysis, chosen so *every* model stays within the user-set accuracy bound while providing memory savings.

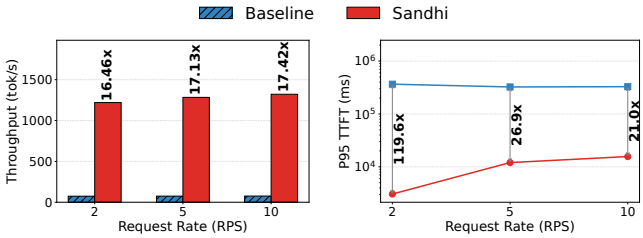


Figure 8. Eliminating offloading for two Llama-3.1-8B models on 1×A100-40GB. Baseline requires offloading, while SANDHI fits entirely in GPU memory, achieving 17× throughput and 120× TTFT improvement.

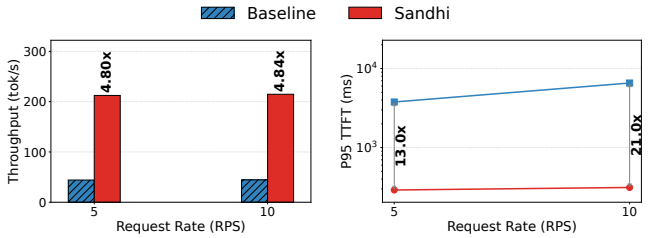


Figure 9. Reducing offload volume for three Qwen3-32B models on 2×A100-80GB. Both configurations require offloading, but SANDHI reduces transferred volume, improving Qwen3's throughput by 4.8× and TTFT by 21×.

5.4 Operational Benefits of Memory Savings

Reducing Infrastructure Costs: SANDHI's memory savings translate directly into system-level benefits. When throughput requirements are fixed, they can reduce the number of GPUs needed for deployment. For example, three Qwen-32B models require 2×H200-141GB with independent serving, whereas SANDHI's 49.8% memory reduction allows the same deployment to fit on 1×H200-141GB. This halves the infrastructure cost while maintaining serving capacity.

Eliminating Offloading: When weights exceed GPU capacity, serving systems offload to CPU memory, adding PCIe transfer latency to every forward pass. SANDHI eliminates this overhead as seen in Fig 8, which shows two Llama-3.1-8B models (SafetyGuard and Multi-Truth) on 1×A100-40GB: the baseline's memory footprint is more than 40GB, which requires offloading. With SANDHI, the merged models fit in the GPU memory, eliminating the need for offloading. The result is a 17× throughput improvement (~75 to ~1300 tok/s) and a 120× TTFT reduction, as no weights traverse PCIe.

Reducing Offloading Overhead: When offloading is unavoidable, SANDHI still reduces transferred volume. Fig 9 shows the case of three Qwen3-32B models on 2×A100-80GB: even SANDHI's merged footprint exceeds capacity, but the reduced offload volume improves Qwen3's throughput by 4.8× (45 to 215 tok/s) and TTFT by 21×.

5.5 Navigating the Accuracy-Memory Tradeoff

SANDHI does not commit to a single merged operating point. The MICR stage generates a family of merged variants as shown in Fig 7. Every point on the pareto-frontier ensures all models in the pool show accuracy within the user-set accuracy threshold. Fig 6e shows why this adaptation matters. In the 9-model deployment, DeepSeek-7B, which has the largest KV footprint, reaches saturation earliest in the baseline and benefits the most from additional memory, achieving up to 2.93× higher throughput and 1052× lower P95 TTFT.

For instance, during workload drift beyond the user-defined threshold (§3.3), SANDHI selects a more memory-efficient configuration (by saving 1.5GB at cost of ≤ 2% accuracy drop) by swapping between merge variants in a model-pool with two DeepSeek-7B. The fetching cost of additional weight

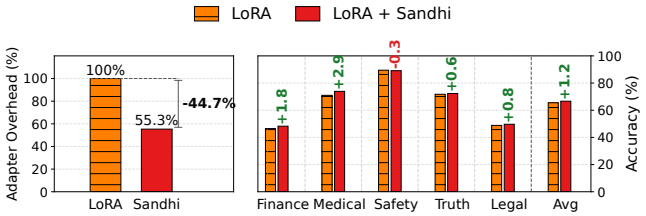


Figure 10. LoRA and SANDHI comparison for Llama models shows that SANDHI provides 44% memory savings on top of the LoRA adapters by merging them while maintaining accuracy.

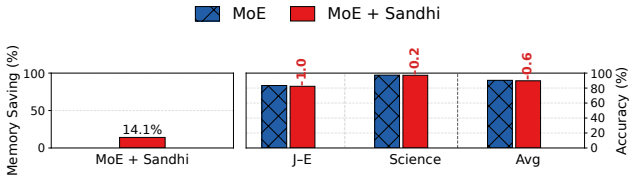


Figure 11. Accuracy & savings with MoE. SANDHI determines mergeable components (including experts) across models reducing 14% (>16 GB) model memory footprint.

is 31ms and is outside of critical path, and atomic pointer-swap takes 0.61ms. Therefore, SANDHI allows for navigating between merged variants while ensuring that P99 TTFT/T-POT remains within 2% of baseline. When that extra memory is no longer needed, SANDHI moves to a point that *improves* accuracy. Thus, *rather than* statically trading accuracy for memory, SANDHI dynamically navigates the frontier, using savings only when the workload benefits from the reclaimed memory by converting it into improved accuracy (§3.3).

5.6 Compatibility with LoRA

SANDHI's component-level merging is complementary to LoRA-based serving; operators can apply LoRA to reduce per-model memory through low-rank adapters, and layer SANDHI on top to share full-rank components across the adapter-augmented models, compounding the savings. Fig 10 reports memory savings and accuracy when both techniques are active. Rather than interfering, the combination improves accuracy with LoRA by +1.2% in average. The result confirms that the two techniques target different axes of redundancy; LoRA compresses individual layers through low-rank factorization, while SANDHI eliminates duplication across models and compose without conflict.

5.7 Compatibility with Quantization

SANDHI's merging is equally complementary to quantization. To validate this, we apply SANDHI to five Llama-3.1-8B fine-tuned models quantized to 8-bit precision under the W8A8 scheme using FP8_BLOCK [16, 37]. Fig 12 shows that we are able to save on top of quantization while the accuracy *improves* by +1.3% on average. As with LoRA, the two techniques address orthogonal sources of memory pressure; quantization reduces per-parameter storage count, while SANDHI reduces parameter count through sharing.

5.8 Compatibility with Mixture-of-Experts

Mixture of expert models replace the feed-forward decoder layers with specialized "experts". Prior works on MoE merging have focused on merging or pruning similar experts *within* the model [8, 43]. Traditional merging techniques require artificially ordering experts in a layer to create mergeable pairs across models. Since SANDHI doesn't rely on pair-wise analysis to merge, it doesn't need to enforce a random ordering. In order to scale across MoE sizes, SANDHI applies noise at the 'expert-layer' granularity while performing magnitude clustering at the 'per-expert' granularity. We show SANDHI on two Qwen3-30B-A3B [72] fine-tunes: MegaScience [17], for scientific reasoning, and Swallow [21], for Japanese–English translation. For evaluation, we use SciQ [36] and xwinograd_jp [53] datasets. Fig 11 shows that SANDHI *atop* MoE, reduces the memory footprint by 14.1% (16.32 GB) while incurring minimal effect on average accuracy (-0.62%). Any meaningful batch size forces offloading. SANDHI reduces footprint to 48GB, leaving 32GB for KV cache. The entire deployment fits in GPU memory, eliminating offloading.

5.9 Ablation Studies

First, we ablate the core design choice of SANDHI: fine-grained, component-level merging §3, by comparing it against a layer-level merging on a pool of five Llama-3-8B models as shown in §5.2. We evaluate the effectiveness of using component-level granularity in SANDHI. Under this restriction, merging at the coarse-grained, per-layer granularity achieves up to 24.9% memory savings, with an average accuracy reduction of −1.9% across models. In contrast, SANDHI's component-level design boosts memory savings by +10.3% to 35.2% while improving average accuracy change by 2.43% to +0.53%, showing the importance of component-level analysis.

Second, we evaluate the profiler's ability to identify mergeable components §3.1 by measuring how often a proposed cluster gets accepted (known as CAR: Cluster Acceptance Rate) by the MICR pipeline. We consider three component selection strategies: using all components, uniformly selecting components across the model depth, and third, selecting the components suggested by the profiler. Since higher CAR leads to more accepted merge steps, it directly enables greater memory savings and a better memory–accuracy trade-off. The profiler-filtered subset achieves 72% CAR, improving over the all-components baseline by 38.05%, which is 18.8% better than the uniform component selection strategy. Moreover, the profiler-guided strategy increases the average accuracy of the merge configurations over the two baselines by +0.6% and +1.84%, respectively. In addition, we measured that, without rollback, profiler incurs an 8% average accuracy-drop (up to 30% for DeepSeek in Fig 5d). These results show that the profiler provides a strong, pool-agnostic signal for component mergeability, substantially improving candidate selection without requiring pairwise comparisons. Omitting each cluster-ordering principle increases rollbacks from 7.7%

to 15.4%/15.4%/23.1% without MLP-first, middle-layer-first, and residual-spacing, respectively. Omitting residual-spacing costs 15% in memory-savings while omitting middle-layer-first costs 2.2 accuracy-points.

Finally, we ablate the scaler that enables merging across model families with different hidden-state and weight distributions §3.2, by comparing the DeepSeek-Coder and DeepSeek-Math model pair. Without the scaler, we are unable to produce any merged variant that satisfies the accuracy target in any of the pools that include these models. Using the scaler (and fine-grained sharing) enables us to reclaim 5.3% of model's memory in all pools with only a -0.6% effect on accuracy. This demonstrates utilizing a magnitude scaler to correct distribution mismatches is essential for enabling merging across homogeneous and heterogeneous model pools.

Runtime. Evaluation and profiling each require at most $2 \times$ `num_layers` $\times$ `eval_time`, where `eval_time` is the duration for a single evaluation (typically 1–5 minutes, depending on the model and benchmark). Profiling and MICR are triggered infrequently (when a new model is added) and costs depend on number-of-GPUs, evaluation dataset size, and number of components the profiler generates. For instance, MICR generates intermediate merged-variants for Llama-pool (Fig 5b) in 20 minutes, running each model in parallel across 2 H200 GPUs, increasing memory-savings from 6.8 GB to 26.8 GB (with accuracy drop going from 0.37% to 0.48%). SANDHI's savings in GPU-hours (freeing up to 50% of the running GPUs (Sec 5.4) or 35% higher throughput (Fig 6a) makes up for the MICR's cost, we believe, SANDHI allows finding the merged configuration with high memory savings and within provided accuracy constraints.

6 Related Work

Fine-tuning: Fine-tuning LLMs for target domains is becoming increasingly popular with the emerging AI workloads, but it remains resource-intensive. PEFT methods [27, 29, 84] reduce the memory and computation needed for fine-tuning.

Evolutionary Optimizations Using Model Merging: Recent work has explored how to systematically combine multiple specialized models into a single, more capable foundation model. M2N2 [3] by Liu et al. proposes a biologically-inspired evolutionary merging mechanism based on *competition* and *attraction*, with the purpose of enabling stable, training-free fusion of diverse LLMs and diffusion models while maintaining or improving task performance. Complementary to this, Akiba et al. introduce *Evolutionary Model Merge*[5] , whose purpose is to automatically search both parameter-space and data-flow–space configurations, discovering cross-domain combinations that outperform their constituents and remove the need for handcrafted merging "recipes". Together, these works position evolutionary search as a promising alternative to manual merging methods, expanding the design space for efficient and scalable multi-model composition.

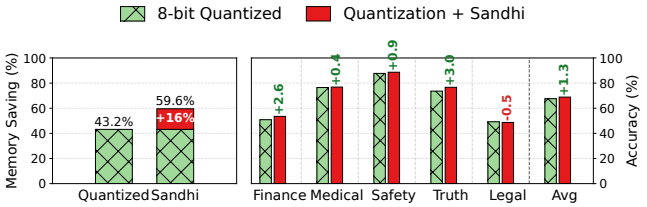


Figure 12. Accuracy of SANDHI with quantized models. SANDHI efficiently reduces model weight footprint by 16% atop quantized variants while ensuring model accuracy is maintained

Multi-Model Serving: Emerging AI workloads [9, 30, 67] with diverse needs resulted in several multi-model serving approaches that support efficient deployment of numerous specialized models. *S-LoRA* [65] enables scalable serving of thousands of LoRA adapters by decoupling the base model from lightweight adapters and using unified GPU memory pooling for both adapters and KV caches. *Punica* [10] introduces a segmented gather kernel to allow cross-model batching across tenants with a single base model, greatly improving memory efficiency and throughput. *dLoRA* [82] builds on this by dynamically switching between merged and unmerged inference paths and orchestrating adapter-request migration to balance load across clusters. Orthogonal to these adapter-based systems, *Mixture-of-Experts* [52] models like the Switch Transformer [19] improve scalability by routing tokens to specialized expert subnetworks, activating only a sparse subset of parameters during inference.

7 Conclusion

We present SANDHI, a system that leverages model merging to alleviate GPU memory pressure. SANDHI combines a trio of novel techniques: (i) noise-based component-level profiling to identify promising merge candidates, (ii) clustering-based iterative merging that adaptively constructs merge configurations guided by these signals, and (iii) a runtime that selects the configuration that maximizes accuracy subject to performance requirements. Across 12 models spanning 3 families and 9 benchmarks, SANDHI reduces GPU memory footprint by up to 49.8%, enabling up to 2.9× higher throughput and up to 1050× lower TTFT.

Acknowledgments

We thank our shepherd, Baris Kasikci, and the reviewers for their valuable feedback. We also thank Xinyu Liang for helping run several evaluations and contributing to the artifact. This work was supported in part by unrestricted research gifts from Adobe and Dolby and by Google Cloud credits provided through the Gemma Academic Program GCP Credit Award and the Gemini Academic Program Award.

References

[1] [n. d.]. NVIDIA Triton Inference Server. https://developer.nvidia.com/nvidia-triton-inference-server.

[2] 2026. CUDA Driver API :: CUDA Toolkit Documentation. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__VA.html. [Accessed 25-02-2026].

[3] João Abrantes, Robert Lange, and Yujin Tang. 2025. Competition and Attraction Improve Model Fusion. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '25)*. ACM, 1217–1225. https://doi.org/10.1145/3712256.3726329

[4] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework For LLM Inference. *ArXiv* abs/2405.05465 (2024). https://api.semanticscholar.org/CorpusID:269635295

[5] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. 2025. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence* 7, 2 (Jan. 2025), 195–204. https://doi.org/10.1038/s42256-024-00975-8

[6] Abraham Atsiwo. 2024. Financial Sentiment Analysis: Leveraging Actual and Synthetic Data for Supervised Fine-tuning. arXiv:2412.09859 [cs.LG] https://arxiv.org/abs/2412.09859

[7] Blend Financial Inc. dba Origin Financial. 2025. How We Built the First AI Financial Advisor with Full-Context Reasoning Regulated by the SEC. https://useorigin.com/resources/blog/technical-overview Blog post; Accessed: 2025-12-11.

[8] Mingyu Cao, Gen Li, Jie Ji, Jiaqi Zhang, Xiaolong Ma, Shiwei Liu, and Lu Yin. 2025. Condense, Don't Just Prune: Enhancing Efficiency and Performance in MoE Layer Pruning. arXiv:2412.00069 [cs.LG] https://arxiv.org/abs/2412.00069

[9] Deming Chen, Alaa Youssef, Ruchi Pendse, André Schleife, Bryan K. Clark, Hendrik F. Hamann, Jingrui He, et al. 2024. Transforming the Hybrid Cloud for Emerging AI Workloads. *CoRR* abs/2411.13239 (2024). https://arxiv.org/abs/2411.13239

[10] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2024. Punica: Multi-Tenant LoRA Serving. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems (MLSys 2024)*, Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (Eds.). mlsys.org, Santa Clara, CA, USA.

[11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[12] Shousheng Jia Xiangzheng Zhang Liang Xu Chenyang Wang, Liang Wen. 2025. Light-IF: Endowing LLMs with Generalizable Reasoning via Preview and Self-Checking for Complex Instruction Following. arXiv:2508.03178 [cs.CL] https://arxiv.org/abs/2508.03178

[13] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[14] Daniel Crankshaw, Gur-Eyal Sela, Corey Zumar, Xiangxi Mo, Joseph E. Gonzalez, Ion Stoica, and Alexey Tumanov. 2020. InferLine: ML Prediction Pipeline Provisioning and Management for Tight Latency Objectives. *arXiv preprint arXiv:1812.01776* (2020).

[15] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[16] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen,

Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. (2025). arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[17] Run-Ze Fan, Zengzhi Wang, and Pengfei Liu. 2025. MegaScience: Pushing the Frontiers of Post-Training Datasets for Science Reasoning. *arXiv preprint arXiv:2507.16812* (2025). https://arxiv.org/abs/2507.16812

[18] Emmett Fear. 2025. A100 GPU Cloud Comparison: Pricing, Performance & Top Providers. https://www.runpod.io/articles/comparison/a100-cloud-comparison. Accessed: 2025-12-11.

[19] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[20] Blanca Calvo Figueras, Eneko Sagarzazu, Julen Etxaniz, Jeremy Barnes, Pablo Gamallo, Iria De Dios Flores, and Rodrigo Agerri. 2025. Truth Knows No Language: Evaluating Truthfulness Beyond English. arXiv:2502.09387 [cs.CL] https://arxiv.org/abs/2502.09387

[21] Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Hiroki Iida, Masanari Ohi, Kakeru Hattori, Hirai Shota, Sakae Mizuki, Rio Yokota, and Naoaki Okazaki. 2024. Continual Pre-Training for Cross-Lingual LLM Adaptation: Enhancing Japanese Language Capabilities. In *First Conference on Language Modeling*.

[22] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation. https://doi.org/10.5281/zenodo.10256836

[23] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vladimir Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. 2024. Arcee's MergeKit: A Toolkit for Merging Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 477–485. https://doi.org/10.18653/v1/2024.emnlp-industry.36

[24] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory

Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[25] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. 2024. The Unreasonable Ineffectiveness of the Deeper Layers. *ArXiv* abs/2403.17887 (2024). https://api.semanticscholar.org/CorpusID:268692211

[26] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] https://arxiv.org/abs/2401.14196

[27] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=lIsCS8b6zj

[28] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. arXiv:2009.03300 [cs.CY] https://arxiv.org/abs/2009.03300

[29] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9

[30] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, et al. 2025. DeepServe: Serverless Large Language Model Serving at Scale. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association. https://www.usenix.org/conference/atc25/presentation/hu-junhao

[31] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[32] Gabriel Ilharco et al. 2022. Editing Models with Task Arithmetic. *arXiv preprint arXiv:2212.04089* (2022).

[33] Shashwat Jaiswal, Shrikara Arun, Anjaly Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renée St. Amant, Chetan Bansal, Victor Ruhle, and Josep Torrellas. 2025. Serving Heterogeneous LoRA Adapters in Distributed LLM Inference Systems. *ArXiv* abs/2511.22880 (2025). https://api.semanticscholar.org/CorpusID:283438836

[34] Fucheng Jia, Zewen Wu, Shiqi Jiang, Huiqiang Jiang, Qianxi Zhang, Yuqing Yang, Yunxin Liu, Ju Ren, Deyu Zhang, and Ting Cao. 2025. Scaling Up On-Device LLMs via Active-Weight Swapping Between DRAM and Flash. *ArXiv* abs/2504.08378 (2025). https://api.semanticscholar.org/CorpusID:277741295

[35] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences* 11, 14 (2021), 6421.

[36] Matt Gardner Johannes Welbl, Nelson F. Liu. 2017. Crowdsourcing Multiple Choice Science Questions. *arXiv:1707.06209v1*.

[37] Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. 2025. "Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization. (2025). arXiv:2411.02355 [cs.LG] https://arxiv.org/abs/2411.02355

[38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. https://doi.org/10.1145/3600006.3613165

[39] Thinking Machines Lab. 2025. Announcing Tinker. https://thinkingmachines.ai/blog/announcing-tinker/ Accessed: 2025-12-11.

[40] Kunfeng Lai, Zhenheng Tang, Xinglin Pan, Peijie Dong, Xiang Liu, Haolan Chen, Li Shen, Bo Li, and Xiaowen Chu. 2025. Mediator: Memory-efficient LLM Merging with Less Parameter Conflicts and Uncertainty Based Routing. *ArXiv* abs/2502.04411 (2025). https://api.semanticscholar.org/CorpusID:276236019

[41] Wonhyuk Lee, Youngchol Kim, Yunjin Park, Junhyung Moon, Dongyoung Jeong, and Wanjin Park. 2025. Guard Vector: Beyond English LLM Guardrails with Task-Vector Composition and Streaming-Aware Prefix SFT. arXiv:2509.23381 [cs.CL] https://arxiv.org/abs/2509.23381

[42] Jiamin Li, Le Xu, Hong Xu, and Aditya Akella. 2024. BlockLLM: Multi-tenant Finer-grained Serving for Large Language Models. *ArXiv* abs/2404.18322 (2024). https://api.semanticscholar.org/CorpusID:269448775

[43] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. 2024. Merge, Then Compress: Demystify Efficient SMoE with Hints from Its Routing Policy. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=eFWG9Cy3WK

[44] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 663–679. https://www.usenix.org/conference/osdi23/presentation/li-zhouhan

[45] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 3214–3252. https://doi.org/10.18653/v1/2022.acl-long.229

[46] Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-Decomposed Low-Rank Adaptation. arXiv:2402.09353 [cs.CL] https://arxiv.org/abs/2402.09353

[47] Yuhan Liu, Yuyang Huang, Jiayi Yao, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. 2026. DroidSpeak: Efficient Context Sharing for Multiple-LLM Inference. In *NSDI*. https://www.microsoft.com/en-us/research/publication/droidspeak-kv-cache-sharing-for-efficient-multi-llm-serving/

[48] Zhenyi Lu et al. 2024. Twin-Merging. *arXiv preprint arXiv:2406.15479* (2024).

[49] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. 2024. tinyBenchmarks: evaluating LLMs with fewer examples. *arXiv preprint arXiv:2402.14992* (2024).

[50] Michael Matena and Colin Raffel. 2022. Merging Models with Fisher-Weighted Averaging. *NeurIPS* (2022).

[51] Microsoft Corporation. 2025. Microsoft 365 Copilot Tuning overview (preview). https://learn.microsoft.com/en-us/copilot/microsoft-365/copilot-tuning-overview. Last updated: 2025-10-27; Accessed: 2025-12-11.

[52] Siyuan Mu and Sen Lin. 2025. A Comprehensive Survey of Mixture-of-Experts: Algorithms, Theory, and Applications. arXiv:2503.07137 [cs.LG] https://arxiv.org/abs/2503.07137

[53] Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M Saiful Bari, Sheng Shen, Zheng-Xin Yong, Hailey Schoelkopf, Xiangru Tang, Dragomir Radev, Alham Fikri Aji, Khalid Almubarak, Samuel Albanie, Zaid Alyafeai, Albert Webson, Edward Raff, and Colin Raffel. 2022. Crosslingual Generalization through Multitask Finetuning. arXiv:2211.01786 [cs.CL]

[54] mukaj. 2024. Llama-3.1-Hawkish-8B v1. https://huggingface.co/mukaj/Llama-3.1-Hawkish-8B. Hugging Face model card (updated Oct 26, 2024), accessed 2026-03-02.

[55] OpenAI. 2025. Fine-tune a model — Model optimization. https://platform.openai.com/docs/guides/model-optimization#fine-tune-a-model. Accessed: 2025-12-11.

[56] OpenMedZoo. 2025. MedGo (Revision 640a2e2). https://doi.org/10.57967/hf/7024

[57] Arthi Padmanabhan, Neil Agarwal, Anand Iyer, Ganesh Ananthanarayanan, Yuanchao Shu, Nikolaos Karianakis, Guoqing Harry Xu, and Ravi Netravali. 2022. GEMEL: Model Merging for Memory-Efficient, Real-Time Video Analytics at the Edge. In *Symposium on Networked Systems Design and Implementation*. https://api.semanticscholar.org/CorpusID:246035830

[58] Maziyar Panahi. 2024. calme-2.3-legalkit-8b: A French Legal Domain Fine-tune of Llama-3.1. https://huggingface.co/MaziyarPanahi/calme-2.3-legalkit-8b. Accessed: 2025-11-01.

[59] Jon Porter. 2023. ChatGPT continues to be one of the fastest-growing services ever. https://www.theverge.com/2023/11/6/23948386/chatgpt-active-user-count-openai-developer-conference.

[60] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Blog* (2019).

[61] Varun Rao, Youran Sun, Mahendra Kumar, Tejas Mutneja, Agastya Mukherjee, and Haizhao Yang. 2025. LLMs Meet Finance: Fine-Tuning Foundation Models for the Open FinLLM Leaderboard. arXiv:2504.13125 [cs.CL] https://arxiv.org/abs/2504.13125

[62] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. INFaaS: A Model-less and Managed Inference Serving System. In *USENIX Annual Technical Conference (ATC)*.

[63] John Schulman and Thinking Machines Lab. 2025. LoRA Without Regret. *Thinking Machines Lab: Connectionism* (2025). https://doi.org/10.64434/tml.20250929 https://thinkingmachines.ai/blog/lora/.

[64] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2023. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *arXiv preprint arXiv:2311.03285* (2023).

[65] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2024. S-LoRA: Scalable Serving of Thousands of LoRA Adapters. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems (MLSys 2024)*, Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (Eds.). mlsys.org, Santa Clara, CA, USA.

[66] Reece Shuttleworth, Jacob Andreas, Antonio Torralba, and Pratyusha Sharma. 2024. LoRA vs Full Fine-tuning: An Illusion of Equivalence. *ArXiv* abs/2410.21228 (2024). https://api.semanticscholar.org/CorpusID:273654311

[67] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, and Ricardo Bianchini. 2025. Rearchitecting Datacenter Lifecycle for AI: A TCO-Driven Framework. *arXiv preprint arXiv:2509.26534* (2025). https://arxiv.org/abs/2509.26534

[68] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2024. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (2024), 1348–1362. https://api.semanticscholar.org/CorpusID:271600544

[69] Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Shang-Wen Li, Wen tau Yih, Jason E. Weston, and Xian Li. 2024. Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM. *ArXiv* abs/2403.07816 (2024). https://api.semanticscholar.org/CorpusID:268363969

[70] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. *ArXiv* abs/2406.03243 (2024). https://api.semanticscholar.org/CorpusID:270258417

[71] T-Tech. 2025. T-pro-it-2.0: A Qwen 3-based Instruction-tuned Model with Reasoning Capabilities. https://huggingface.co/t-tech/T-pro-it-2.0. Accessed: 2025-11-01.

[72] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[73] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient

Foundation Language Models. arXiv:2302.13971 [cs.CL] https://arxiv.org/abs/2302.13971

[74] us4. 2024. Fin-LLaMA 3.1 8B - Fine-tuned on Financial News. https://huggingface.co/us4/fin-llama3.1-8b.

[75] Bhoomit Vasani, Jack G. M. Fitzgerald, Anjie Fang, and Sushmit Vaish. 2025. PHLoRA: data-free Post-hoc Low-Rank Adapter extraction from full-rank checkpoint. In *IJCNLP-AACL*. https://api.semanticscholar.org/CorpusID:281314959

[76] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*.

[77] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, Tal Linzen, Grzegorz Chrupała, and Afra Alishahi (Eds.). Association for Computational Linguistics, Brussels, Belgium, 353–355. https://doi.org/10.18653/v1/W18-5446

[78] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574* (2024).

[79] Zhibin Wang, Zhonghui Zhang, Yuhang Zhou, Zibo Wang, Mo Zhou, Peng Jiang, Weilin Cai, Chengying Huan, Rong Gu, Sheng Zhong, and Chen Tian. 2025. Accelerating Mixture-of-Experts Inference by Hiding Offloading Latency with Speculative Decoding. *ArXiv* abs/2508.21706 (2025). https://api.semanticscholar.org/CorpusID:280985081

[80] Joe H. Ward. 1963. Hierarchical Grouping to Optimize an Objective Function. *J. Amer. Statist. Assoc.* 58 (1963), 236–244. https://api.semanticscholar.org/CorpusID:32863022

[81] Mitchell Wortsman et al. 2022. Model Soups. In *ICML*.

[82] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, USA, 911–927.

[83] Jiarong Xing, Yifan Qiao, Simon Mo, Xingqi Cui, Gur-Eyal Sela, Yang Zhou, Joseph Gonzalez, and Ion Stoica. 2025. Towards Efficient and Practical GPU Multitasking in the Era of LLM. *arXiv preprint arXiv:2508.08448* (2025).

[84] Lingling Xu, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. 2023. Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment. *arXiv preprint arXiv:2312.12148* (2023).

[85] Prateek Yadav et al. 2024. TIES-Merging. In *NeurIPS*.

[86] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[87] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. arXiv:2409.12122 [cs.CL] https://arxiv.org/abs/2409.12122

[88] Enneng Yang et al. 2023. Adamerging. *arXiv preprint arXiv:2310.02575* (2023).

[89] Enneng Yang et al. 2024. Model Merging in LLMs. *arXiv preprint arXiv:2408.07666* (2024).

[90] Huan Yang, Renji Zhang, Mingzhe Huang, Weijun Wang, Yin Tang, Yuanchun Li, Yunxin Liu, and Deyu Zhang. 2025. KVShare: An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse. https://api.semanticscholar.org/CorpusID:277244216

[91] Xiaozhe Yao, Qinghao Hu, and Ana Klimovic. 2025. DeltaZip: Efficient Serving of Multiple Full-Model-Tuned LLMs. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 110–127. https://doi.org/10.1145/3689031.3717468

[92] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, et al. 2025. Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving. *arXiv preprint arXiv:2505.04021* (2025).

[93] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. 2025. Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving. arXiv:2505.04021 [cs.DC] https://arxiv.org/abs/2505.04021

[94] Jianzhang Zhang, Jialong Zhou, and Chuang Liu. 2025. OR-Toolformer: Modeling and Solving Operations Research Problems with Tool Augmented Large Language Models. arXiv:2510.01253 [cs.AI] https://arxiv.org/abs/2510.01253

[95] Kaiyan Zhang, Ning Ding, Biqing Qi, Sihang Zeng, Haoxin Li, Xuekai Zhu, Zhang-Ren Chen, and Bowen Zhou. 2024. UltraMedical: Building Specialized Generalists in Biomedicine. https://github.com/TsinghuaC3I/UltraMedical.

[96] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models using SGLang. arXiv:2312.07104 [cs.AI]

[97] Qihao Zhu Runxin Xu Junxiao Song Mingchuan Zhang Y.K. Li Y. Wu Daya Guo Zhihong Shao, Peiyi Wang. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. https://arxiv.org/abs/2402.03300